Prerequisites

Splunk's hands-on workshops are delivered via the Splunk Show portal - **you will need a splunk.com account in order to access this**.
If you don't already have a Splunk.com account, please create one here before proceeding with the rest of the workshop.

If you don't already have a Splunk.com account, please create one here before proceeding with the rest of the workshop.

Troubleshooting Connectivity

If you experience connectivity issues with accessing either your workshop environment or the event page, please try the following troubleshooting steps. If you still experience issues please reach out to the team running your workshop.

- **Use Google Chrome** (if you're not already)
- If the event page (i.e. https://splunk.show/…) didn't load when you clicked on the link, try **refreshing the page**
- **Disconnect from VPN** (if you're using one)
- **Clear your browser cache and restart your browser** (if using Google Chrome, go to: Settings > Privacy and security > Delete browsing data)
- **Try using private/incognito browsing mode** to rule out any caching issues

Need help?
If you're still experiencing issues please ask a member of the onsite support team for assistance and they'll help get you up and running!

Description

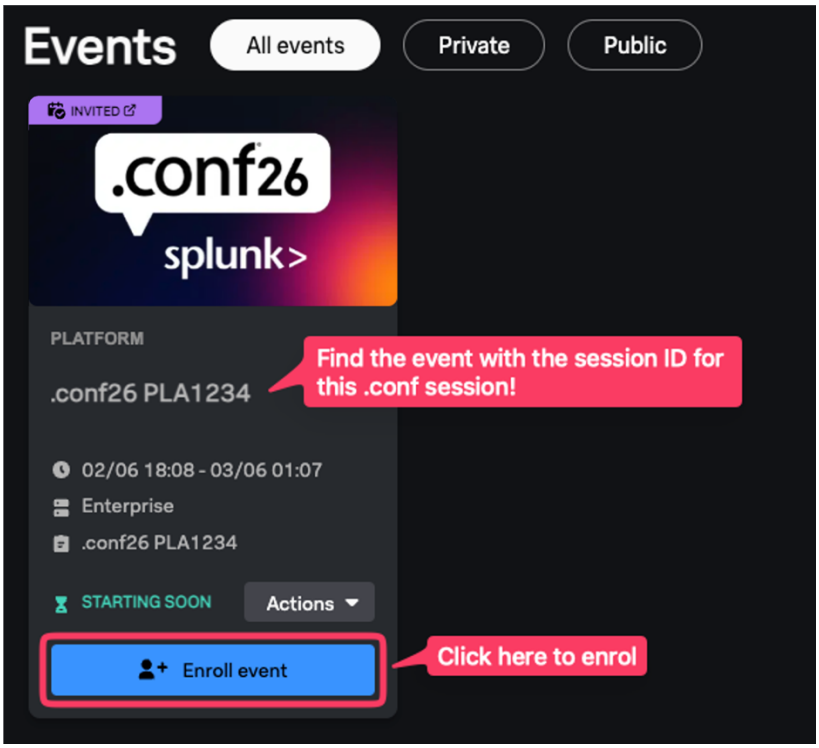
In this exercise, you will access your lab environment using our Splunk Show portal.

Login to Splunk Show

You should have received an invite email from **Splunk Show** containing a link to a workshop event ("**https://splunk.show/…**")

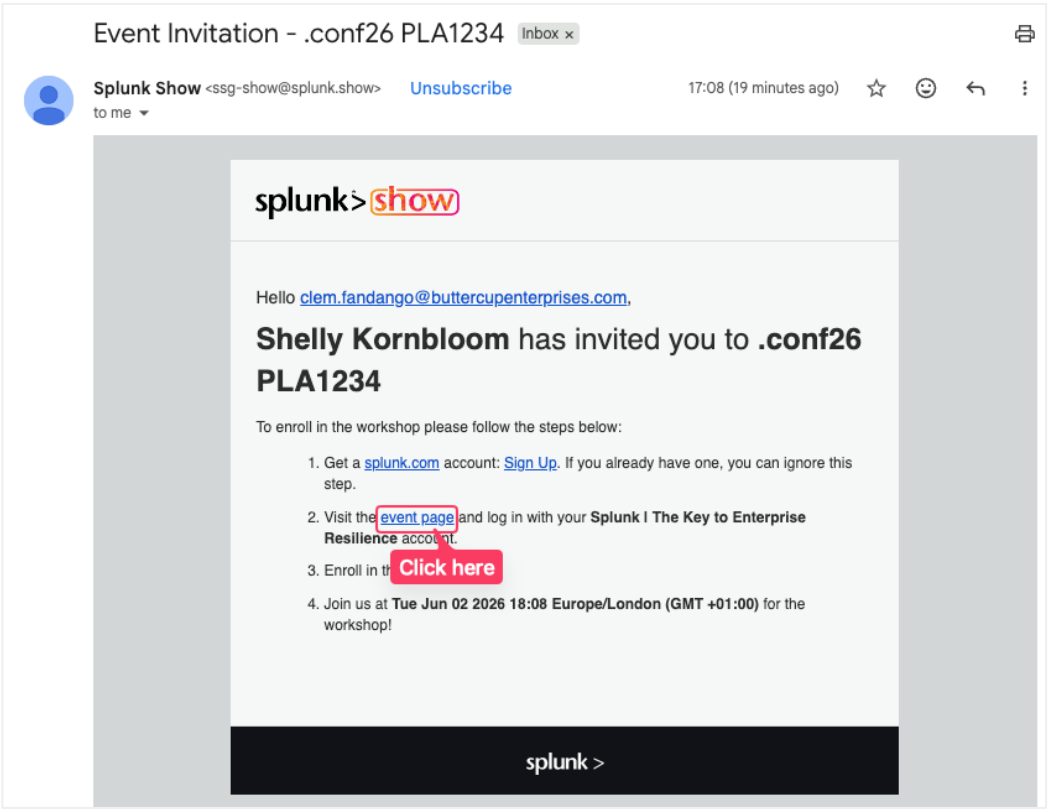
❌ If you DON'T have an email from Splunk Show:

1. Please check your spam folder.
2. If you still can't find the email please visit https://show.splunk.com and log in with your splunk.com account.
3. Find the event for this session (look for the session ID, such as PLA1234, SEC5678, etc.) and click on **Enroll event**.

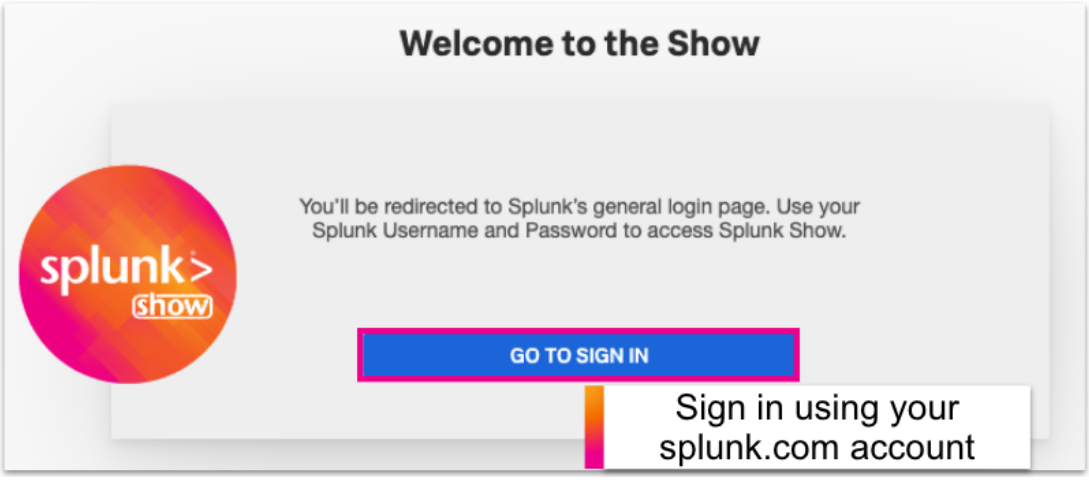


✅ **If you DO have an email from Splunk Show:**

1. Open the email and click on the event link.

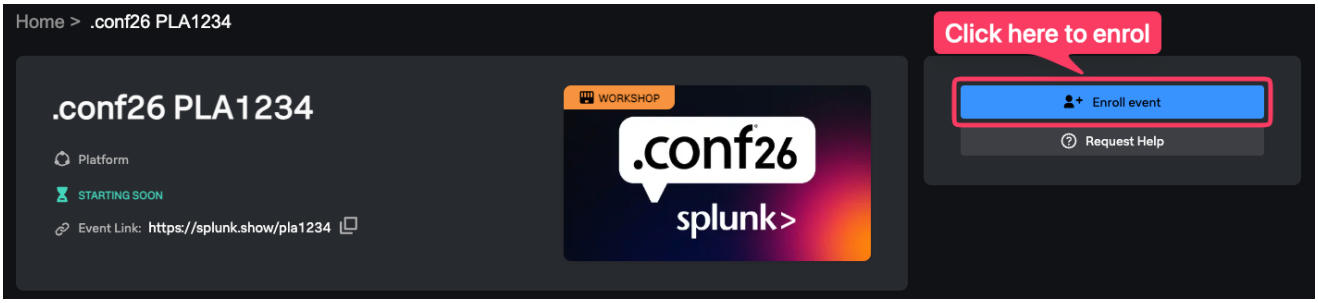


2. On the **Welcome to the Show** page click on **GO TO SIGN IN** and log in using your **Splunk.com account**.



🔑 **Don't have a Splunk.com Account?**
If you don't already have a Splunk.com account, don't worry - it only takes a few minutes to create one! Please create one here.

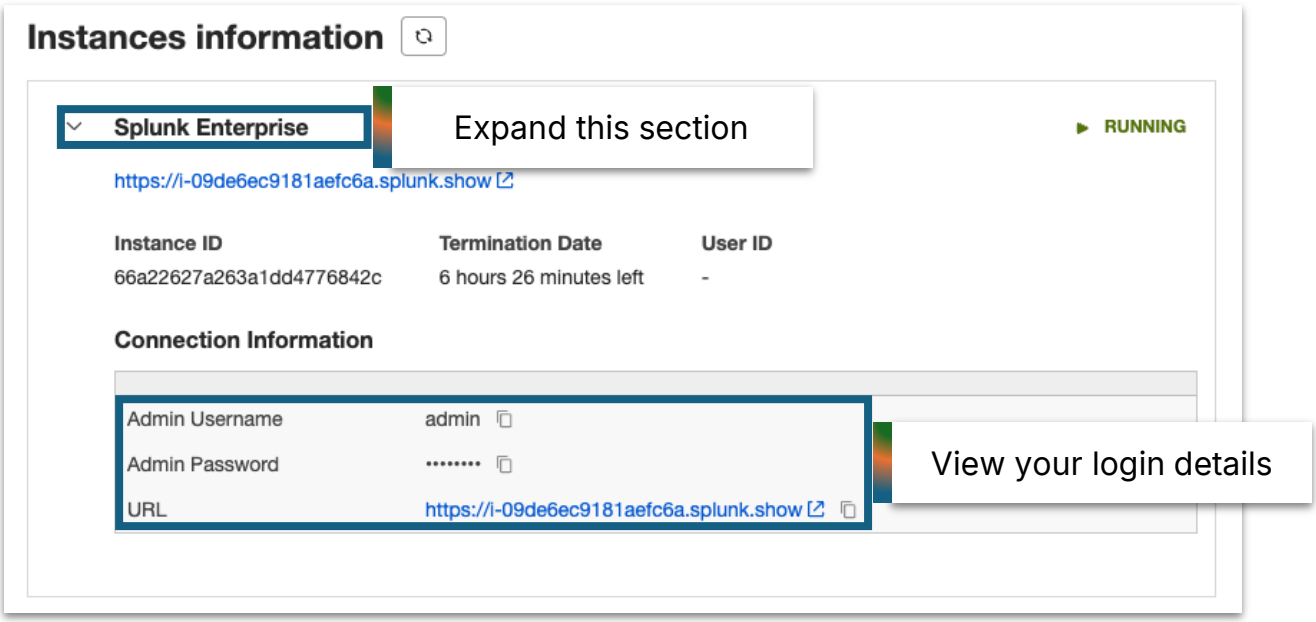
3. Once logged in you should see the event page. Click on **Enroll event**.



Need help enrolling?
Please ask a member of the onsite support team for assistance.

Finding Your Instance Details:

1. Scroll down the page to the **Instances information** section. Here you will find information on how to access your environment(s) along with the login details.



Note: The screenshot above shows 'Splunk Enterprise' but it may be called something else depending on the environments used in this hands-on session.

OPERATION: LAST CALL

Participant Workbook

A hands-on Living-off-the-Land threat hunt · Splunk .conf26 · 90 minutes

---

You are Alice Bluebird, a threat hunter at **Barrel And Signal**. An EDR alert on certutil.exe was closed as a false positive last week, and you don't buy it. Over the next 90 minutes you'll hunt the **Sable Vector** intrusion phase by phase, using only Splunk Enterprise, Sysmon, and Windows Event Logs. Everything the attacker did, they did with tools that ship with Windows.

How to use this workbook

- **Code blocks** are copy-paste ready. Type them for the reps, or paste if you're moving fast.
- **What you should find** tells you the answer so you can self-check — this is a guided hunt, not a competition.
- **Checkpoint (✅)** is a quick question to confirm you're on track.
- **Stretch (🔍)** is optional — for when you finish early.

> ℹ️ **Tip** Set your time picker to **`All time`** for every search today. The intrusion is historical — a narrow window hides it.

Your pod and the data

Each of you has your own Splunk instance with your own copy of the Operation: Last Call dataset. Open your pod using the URL and credentials on your table card, then confirm the data is present:

```
| tstats count where index=* by index, host
| sort index host
```

> **You should see** Four indexes — **`sysmon`**, **`wineventlog`**, **`dns`**, **`exchange`** — across five hosts. No results? Flag an instructor before Exercise 0.

The five hosts

- **`BS-WS-OPS-07`** Ops workstation — patient zero. Phishing lure, execution, C2, credential theft.
- **`BS-FS-01`** File server — the prize. Finance data staged & exfiltrated. Also firewall/physical logs.
- **`BS-WS-EXEC-02`** Executive workstation — a lateral-movement target.
- **`BS-DC-01`** Domain controller — authentication, DNS, and Windows Event Log for the domain.
- **`BS-MAIL-01`** Mail/Exchange — where the phish arrived (exchange index carries mail + IIS).

How we'll hunt: PEAK

We follow Splunk's PEAK framework — **Prepare** (baseline your normal), **Execute** (four hunts), and **Act** (turn a hunt into a detection). **Knowledge** from each step sharpens the next.

Exercise 0: Baseline — know your normal

PEAK Prepare · Baseline hunt · 10 min

Objective

Before you can spot evil, you need to know what these Windows binaries normally do in this environment. Establish that baseline first.

Background

Sable Vector lived off the land — every tool they used is legitimate and runs in normal operations too. The only way to separate attacker from admin is to know the usual footprint: which LOLBins run, on which hosts, spawned by which parents.

Run the hunt

```
index=sysmon EventCode=1
| eval proc=lower(Image), parent=lower(ParentImage)
| where match(proc, "(rundll32|regsvr32|msiexec|certutil|bitsadmin|mshta|wmic|powershell)\.exe")
| stats count values(parent) as parents by proc, host
| sort -count
```

What you should find - A short, predictable list. Most of these binaries barely run and are spawned by expected parents (svchost, services, explorer for admin tasks). That quiet baseline is the point - anything outside it in the next four hunts is your lead.

✅ **Checkpoint** Which host shows the most varied LOLBin activity, and does anything already look out of place for a workstation?

🔍 **Stretch** Add `| where count < 5` to surface the rarest process activity - the long tail is where hunts begin.

Exercise 1: Hunt the initial execution

Phase 1–2 · T1059 / T1204 · 15 min

Objective

Find where the intrusion began: an application that should never launch a system utility, doing exactly that.

Background

The phishing lure landed on BS-WS-OPS-07. When the user opened it, the document's script engine (wscript.exe) launched a living-off-the-land binary to pull down the next stage. Signature tools saw two trusted Microsoft binaries and shrugged. You're going to hunt the relationship instead.

Run the hunt

```
index=sysmon EventCode=1
| eval parent=lower(ParentImage), proc=lower(Image)
| where match(parent,"(powershell|wscript|cscript|mshta)\.exe")
    AND match(proc,"(rundll32|regsvr32|certutil|bitsadmin|msiexec)\.exe")
| stats count by _time, host, ParentImage, Image, CommandLine
| sort -count
```

What you should find On **`BS-WS-OPS-07`**: **`powershell.exe`** spawning **`rundll32.exe, bitsadmin.exe and certutil.exe`** — the lure detonating. Read the CommandLine: the arguments reveal what was loaded and from where.

✅ **Checkpoint** What is the full CommandLine on the child process, and what does it tell you the attacker was doing at this stage?

🔍 **Stretch** This query is technique-agnostic. Look for `explorer.exe → powershell.exe` — that's the pattern a ClickFix / paste-and-run lure would leave. Same hunt, modern entry point.

Exercise 2: Find the C2 rhythm

Phase 3 · T1071 / T1572 · 15 min

Objective

Confirm the compromised host is calling home by finding a suspiciously regular beacon in DNS traffic.

Background

Once the payload ran, it established command-and-control. Beacons check in on a timer - often with a little random jitter to look human - but the underlying regularity gives them away. You'll compute the time gap between repeated DNS queries to the same domain and look for a steady heartbeat.

Run the hunt

Part A:

First, find the anomalous domains. Look for addresses that are queried by *very few* hosts, that stick out from the noise floor.

```
index=dns
| search _raw="*Rcv*"
| stats count dc(src) as hosts values(src) as src_list by query
| where count > 10 AND hosts <= 2
| sort -count
```

Part B:

Once found, consider bucketing the data so that you can separate possible stages of attack. There might be a recon phase, a persistence phase and an exfil phase all living in your dataset.

```
index=dns valdoris src=10.10.7.208
| search _raw="*Rcv*"
| sort 0 _time
| streamstats current=f last(_time) as prev by src
| eval interval = _time - prev
| where interval > 60 AND interval < 600
| eval bucket = round(interval/30)*30
| stats count by bucket
| sort -count
```

Part C:

You have found a spike, now look to see whether it is a disciplined beacon, something that would set it apart from normal human or machine behaviour.

```
index=dns valdoris src=10.10.7.208
| search _raw="*Rcv*"
| sort 0 _time
| streamstats current=f last(_time) as prev by src
| eval interval = _time - prev
| where interval >= 200 AND interval <= 290
| stats count avg(interval) as avg_beacon stdev(interval) as jitter by src
| eval jitter_pct = round(jitter/avg_beacon*100,1)
```

What you should find - A non-corporate domain queried from **`BS-WS-OPS-07`** roughly every 240 seconds with low jitter. **`avg_beacon ≈ 240`** and **`jitter_pct < 20`** is a strong C2 positive.

✅ **Checkpoint** What domain is the beacon calling, and what is its average interval? Would you have spotted it by eyeballing raw DNS logs?

🔍 **Stretch** Real C2 uses long, random sleeps and HTTPS. Lower the `count > 15` threshold — how few callbacks can you still detect the pattern with before it drowns in noise?

Exercise 3: Hunt the access

Phase 4–5 · T1003.001 / T1547 / T1053 · 15 min

Objective

Answer two questions: did anyone read credentials out of LSASS memory, and did they leave themselves a way back in?

Background

With code running and C2 established, Sable Vector dumped credentials from LSASS and set up persistence. The dump used comsvcs.dll via rundll32 — but you won't hunt for rundll32, because that would miss ProcDump, Task Manager, and every other dumper. Instead you hunt the behaviour: a process opening a powerful handle to lsass.exe.

Run the hunt

```
index=sysmon EventCode=10 TargetImage="*\\lsass.exe"
| where GrantedAccess IN ("0x1010","0x1410",
    "0x1438","0x143a","0x1f1fff","0x1FFFFF")
| search NOT SourceImage IN
    ("*\\MsMpEng.exe","*\\CSFalconService.exe")
| stats count by ComputerName, SourceImage,
    GrantedAccess, CallTrace
```

What you should find On **`BS-WS-OPS-07`**: **`rundll32.exe`** opening a handle to **`lsass.exe`** (the comsvcs MiniDump technique). Masks: `0x1010` Mimikatz, `0x1410/0x1f1fff` ProcDump/comsvcs, `0x1ffffff` full access.

✅ **Checkpoint** Which GrantedAccess mask did the attacker use - and what does the CallTrace field tell you that the mask alone doesn't?

🔍 **Stretch** Now find the persistence they left behind. Run `index=sysmon EventCode=13 | where match(TargetObject,"CurrentVersion\\Run")` for the Run key, and `index=wineventlog EventCode=4698` for the scheduled task. Two ways back in.

Exercise 4: Reconstruct the whole intrusion

Phase 6–8 · T1021 / T1074 / T1560 / T1048 · 13 min

Objective

Stitch lateral movement, data staging, and exfiltration into a single chronological timeline with one search.

Background

In the final phases, Sable Vector used explicit credentials to reach the file server (BS-FS-01), created a remote scheduled task, staged finance data with robocopy, encoded it with certutil, and exfil'd it with bitsadmin. Rather than hunt each separately, one multisearch labels and merges them into an ordered story.

Run the hunt

```
| union
    [ search index=exchange sourcetype=ms:exchange:journal sender="accounts@barrelandsignal-
support.net"
      | sort 0 _time | head 1
      | eval phase="1 · Initial Access (phish)", detail="From ".sender." — ".subject, host="BS-MAIL-
01" ]
    [ search index=sysmon EventCode=1 Image="*\\rundll32.exe" CommandLine="*sc_update.dll*" host="BS-
WS-OPS-07*"
      | sort 0 _time | head 1
```

```
      | eval phase="2 · Execution (implant launch)", detail=CommandLine ]
    [ search index=dns "officeupdate-cdn" "Rcv" src=10.10.7.208
      | sort 0 _time | head 1
      | eval _time=_time-14400, phase="3 · Command & Control (first beacon)", detail=query, host="BS-WS-OPS-07 (10.10.7.208)" ]
    [ search index=sysmon EventCode=1 Image="*\\reg.exe" CommandLine="*CurrentVersion\\Run*" host="BS-WS-OPS-07*"
      | sort 0 _time | head 1
      | eval phase="4 · Persistence (Run key)", detail=CommandLine ]
    [ search index=sysmon EventCode=10 TargetImage="*\\lsass.exe" host="BS-WS-OPS-07*"
      | sort 0 _time | head 1
      | eval phase="5 · Credential Access (LSASS)", detail="GrantedAccess ".GrantedAccess." via ".SourceImage ]
    [ search index=wineventlog EventCode=4648 "svc_stillwaterSync" ComputerName="BS-WS-OPS-07*"
      | sort 0 _time | head 1
      | rex field=Message "Target Server Name:\s*(?<target>\S+)"
      | eval _time=_time-14400, phase="6 · Lateral Movement", detail="svc_stillwaterSync -> ".target ]
    [ search index=sysmon EventCode=1 Image="*\\robocopy.exe"
      | sort 0 _time | head 1
      | eval phase="7 · Collection (robocopy)", detail=CommandLine ]
    [ search index=sysmon EventCode=1 Image="*\\certutil.exe" CommandLine="*-encode*"
      | sort 0 _time | head 1
      | eval phase="8 · Exfiltration (certutil encode)", detail=CommandLine ]
    [ search index=sysmon EventCode=1 Image="*\\bitsadmin.exe" CommandLine="*upload*"
      | sort 0 _time | head 1
      | eval phase="8 · Exfiltration (bitsadmin)", detail=CommandLine ]
| eval host=coalesce(Computer, ComputerName, host)
| sort 0 _time
| table _time phase host detail
```

What you should find A chronological chain moving from **BS-WS-OPS-07** to **BS-FS-01**: explicit creds → remote schtasks → robocopy staging → certutil encode → bitsadmin upload. That table is the opening paragraph of your incident report.

✅ **Checkpoint** In what order did the phases occur, and roughly how long did the whole intrusion take from first execution to exfiltration?

🔍 **Stretch** Pick any single branch and imagine it as a scheduled alert. Which one would you deploy first, and why? (Hint: which has the fewest legitimate uses on a file server?)

Exercise 5: Turn your hunt into a detection

PEAK Act · Risk-Based Alerting · 10 min

Objective

Make Exercise 1 repeatable: promote it to a saved detection (post .conf with ES - give it a risk score instead of a raw alert).

Background

A hunt you run once has limited value; a hunt you automate protects you every day. But a single LOLBin event is noisy. Risk-Based Alerting (RBA) is Splunk's answer – with Splunk Enterprise Security you can assign a small risk score to each weak signal, and let them accumulate on a host until they cross a threshold worth paging on. Splunk names living-off-the-land as a headline RBA use case.

Run the hunt

```
// Start from your Exercise 1 search, then add:
| stats count min(_time) as firstTime max(_time) as lastTime
    by ComputerName, ParentImage, Image, CommandLine
```

```
// Then in the UI:
//  1. Save As > Alert (or Correlation Search in ES)
//  2. Schedule it (e.g. every 15 min, real-time-ish)
//  3. If you have access to Splunk ES, add a risk annotation:
//       risk_object = ComputerName, risk_score = 40,
//       threat_object = Image, MITRE = T1059
```

What you should find - A scheduled detection that adds risk to the host rather than paging on every certutil. On its own it's a nudge; combined with the Run-key, task, and LSASS signals from Exercise 3, RBA escalates the host to one high-fidelity finding — and teams see **50–90% fewer alerts**.

✅ **Checkpoint** Why is a risk score better than a straight alert for a noisy signal like this? What other signals from today would you add risk for on the same host?

🔍 **Stretch** Sketch the risk math: if this chain is +40, the Run-key +30, the off-hours task +25, and the LSASS handle +60, what threshold would fire on BS-WS-OPS-07 — and would any single signal alone trip it?

Exercise 6: The Splunk Threat Hunting Workbench

Our instructors will guide you through this section during the session. Pay attention, and reach out to us after the session if you would like access to the Threat Hunting Workbench alpha program.

What you just did

In 80 minutes you reconstructed a 22-hour intrusion from telemetry alone, without relying on a single signature. You baselined normal, hunted the behaviour at each phase, reconstructed the timeline, and promoted a hunt into a detection. The six queries matter less than the method — it transfers to many environments and outlives any specific technique.

Take it home

- **The Hunt Card** — all six queries, event codes, and MITRE IDs on one page. Run them against your own data this week.
- **PEAK Threat Hunting Framework** (Splunk SURGe) — structure your hunts.
- **Essential Guide to Risk-Based Alerting** — turn hunts into high-fidelity findings.
- **BOTS Kiosk on the .conf26 floor** — put today's skills to the test, competitively.

Appendix — key event codes

Source	Code	What it captures
Sysmon	1	Process Create — image, CommandLine, ParentImage, hashes
Sysmon	3	Network Connection — src/dst, port, process
Sysmon	10	Process Access — GrantedAccess mask (LSASS)
Sysmon	11	File Create — staged archives, dump files
Sysmon	13	Registry Value Set — Run keys, service config
Sysmon	22	DNS Query — QueryName, process
WinEvent	4624	Logon (with LogonType)
WinEvent	4648	Explicit Credential Use — net use / runas
WinEvent	4698	Scheduled Task Created
WinEvent	5140	Network Share Access — lateral pivot
WinEvent	4104	PowerShell Script Block